

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: K.Vinay

Roll No.: A24126552086

Date: 21-09-2026

1. TITLE

Implementation of File System Operations using Node.js

2. AIM

To create a Node.js command-line application that performs basic file operations (Create, Read, and Append) using the built-in fs module and captures user input asynchronously.

3. OBJECTIVE

The objectives of this experiment are:

- To use the readline module for interactive CLI prompts.
- To utilize the fs module to manipulate the local file system.
- To write, read, and append to a file asynchronously.
- To handle potential I/O errors through callback functions.

4. THEORY

Node.js provides the fs (File System) module which enables interaction with the file system. Methods such as writeFile, readFile, and appendFile perform asynchronous operations, using callbacks to handle errors and ensure non-blocking execution. The readline module allows Node.js to read standard input stream data interactively from the user.

5. ALGORITHM / PROCEDURE

1. Import the fs and readline modules in Node.js.
2. Create an interface to read from process.stdin and write to process.stdout.
3. Prompt the user to enter a filename and the initial content for the file.
4. Use fs.writeFile() to create the file and store the content.
5. Use fs.readFile() to read back the original content and display it.
6. Prompt the user for additional content to append.
7. Use fs.appendFile() to append the new text to the existing file.
8. Use fs.readFile() to display the final updated content, and close the interface.

6. CODE EXPLANATION

The `require('fs')` module is initialized to manage asynchronous I/O requests. The `readline.createInterface` sets up terminal prompting. Through nested callbacks, the app gathers the filename and file content via `rl.question`. It then uses `fs.writeFile()` to dump the content, reads it back with `fs.readFile()`, requests more content from the user, and uses `fs.appendFile()` to append to the document safely before closing the readline stream.

7. PROGRAM

2_app.js

```
const fs = require("fs");
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question("Enter filename: ", (filename) => {

  rl.question("Enter content: ", (content) => {

    // 1. Create / Write file
    fs.writeFile(filename, content, (err) => {

      if (err) {
        console.log("Error:", err);
        rl.close();
        return;
      }

      console.log("\nFile created successfully!");

      // 2. Read file
      fs.readFile(filename, "utf8", (err, data) => {

        if (err) {
          console.log("Error:", err);
          rl.close();
          return;
        }

        console.log("\nOriginal Content:");
        console.log(data);

        rl.question(
          "\nEnter additional content: ",
          (additionalContent) => {

            // 3. Append content
            fs.appendFile(
              filename,
              "\n" + additionalContent,
              (err) => {

                if (err) {
                  console.log("Error:", err);
                }
              }
            );
          }
        );
      }
    );
  }
);
```

```

        rl.close();
        return;
    }

    console.log("\nContent appended successfully!");

    // 4. Display final content
    fs.readFile(
        filename,
        "utf8",
        (err, finalData) => {

            if (err) {
                console.log("Error:", err);
                rl.close();
                return;
            }

            console.log("\nFinal Content:");
            console.log("-----");
            console.log(finalData);
            console.log("-----");

            rl.close();
        });
    });
});
});
});
});
});

```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Provide filename & content	File created	File created and original content printed correctly	Pass
TC-02	Provide append text	Text appended to file	Final content printed accurately includes appended text	Pass

9. OUTPUT

```
Node.js v24.19.0
PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development> node ".\observations tasks\task 5\2_app.js"
Enter filename: notes.txt
Enter content: i am vinay

File created successfully!

Original Content:
i am vinay

Enter additional content: node js is easy to learn

Content appended successfully!

Final Content:
-----
i am vinay
node js is easy to learn
-----
PS C:\Users\vinay\OneDrive\Desktop\full-stack-web-development> █
```

10. RESULT

Thus, the file system operations involving reading, writing, and appending files were successfully executed asynchronously in Node.js.